

## Contenido

|                                                                                                                                   |    |
|-----------------------------------------------------------------------------------------------------------------------------------|----|
| 1. Sentido del resultado de aprendizaje y visión global de la calidad .....                                                       | 1  |
| 2. Selección de algoritmos según el problema, los datos y las restricciones del proyecto .....                                    | 3  |
| 3. Principios y técnicas básicas de los sistemas inteligentes.....                                                                | 5  |
| 4. Integración de principios fundamentales de la computación en proyectos de IA .....                                             | 6  |
| 5. Desarrollo de sistemas y aplicaciones informáticas que incorporan técnicas inteligentes .....                                  | 8  |
| 6. Técnicas de aprendizaje computacional orientadas a la extracción automática de información en grandes volúmenes de datos ..... | 9  |
| 7. Validación, capacidad de generalización, test y control del sobreajuste .....                                                  | 11 |
| 8. Métricas de evaluación y lectura de la matriz de confusión .....                                                               | 13 |
| 9. Interpretación de resultados, explicabilidad y toma de decisiones.....                                                         | 15 |
| 10. Casos prácticos guiados .....                                                                                                 | 16 |
| 11. Buenas prácticas y errores frecuentes .....                                                                                   | 17 |

## 1. Sentido del resultado de aprendizaje y visión global de la calidad.

### 1.1. Qué significa valorar la calidad de un modelo.

En aprendizaje automático, hablar de calidad no significa únicamente obtener un porcentaje alto de aciertos. Un modelo es de calidad cuando responde de manera fiable al problema para el que ha sido diseñado, cuando mantiene un comportamiento estable al enfrentarse a nuevos datos, cuando su coste computacional es razonable y cuando sus resultados pueden integrarse en una solución real.

Por tanto, la calidad es un concepto **multidimensional**. Incluye la precisión predictiva, pero también la capacidad de generalización, la robustez ante ruido o datos incompletos, la rapidez de entrenamiento e inferencia, la facilidad de mantenimiento, la interpretabilidad y el ajuste al contexto de uso. Un modelo excelente en laboratorio puede ser mediocre en producción si necesita tiempos de respuesta imposibles o si falla cuando cambian ligeramente los datos de entrada.

Esta idea es especialmente importante en entornos profesionales. Las organizaciones no adoptan modelos por ser “interesantes”, sino porque resuelven necesidades concretas: clasificar documentos, predecir demanda, detectar fraude, agrupar clientes, automatizar recomendaciones o extraer patrones de grandes conjuntos de datos. La calidad, entonces, se mide siempre respecto a una finalidad.

### 1.3. La calidad como proceso y no como momento aislado.

Un error muy común consiste en pensar que la calidad se evalúa al final, cuando el modelo ya está entrenado. En realidad, la calidad se construye desde el principio del proyecto. Comienza con una buena definición del problema, continúa con la selección adecuada de los datos, se

consolida mediante validación rigurosa y se comprueba con métricas significativas. Después debe seguir vigilándose en despliegue, porque los datos del mundo real cambian.

Por ello, conviene entender un proyecto de aprendizaje automático como un ciclo. Primero se formula el problema. Después se adquieren y preparan los datos. A continuación se seleccionan modelos, se entrenan, se validan y se comparan. Finalmente, se despliegan y monitorizan. Si alguna fase falla, la evaluación final queda comprometida.

#### Idea clave

La pregunta correcta no es “¿qué modelo da mejor resultado?” sino “¿qué solución ofrece el mejor equilibrio entre rendimiento, generalización, coste, robustez, interpretabilidad y valor práctico?”

### 1.4. Calidad técnica frente a utilidad de negocio o de servicio.

En contextos reales puede ocurrir que el modelo con mejor métrica no sea el mejor para implantar. Un ejemplo clásico es la comparación entre un modelo muy complejo y otro algo menos preciso pero mucho más interpretable. Si una entidad financiera necesita explicar por qué concede o deniega un préstamo, quizá prefiera un modelo ligeramente menos preciso pero más comprensible y auditable.

También sucede lo contrario. En una tarea de visión artificial industrial, una pequeña mejora en la detección de defectos puede compensar de sobra un mayor coste de cómputo, porque evita pérdidas económicas elevadas. Esto demuestra que la valoración de la calidad siempre requiere una lectura contextual, donde entran en juego la finalidad, el impacto, las restricciones y el riesgo.

| Dimensión              | Qué pregunta responde                             | Ejemplo práctico                                             |
|------------------------|---------------------------------------------------|--------------------------------------------------------------|
| Rendimiento predictivo | ¿Acierta lo suficiente?                           | Porcentaje de correos spam bien detectados.                  |
| Generalización         | ¿Se comporta bien con datos nuevos?               | El modelo mantiene resultados con datos de un mes posterior. |
| Robustez               | ¿Tolera ruido, valores faltantes o cambios leves? | No se degrada gravemente ante registros incompletos.         |
| Eficiencia             | ¿Consume tiempo y memoria asumibles?              | Predice en milisegundos para una aplicación web.             |
| Interpretabilidad      | ¿Puede justificarse su decisión?                  | Se explican las variables que influyen en la predicción.     |
| Escalabilidad          | ¿Soporta grandes volúmenes de datos?              | Puede entrenarse o inferir sobre millones de registros.      |

## 2. Selección de algoritmos según el problema, los datos y las restricciones del proyecto.

### 2.1. Empezar por el problema y no por la herramienta.

Elegir un algoritmo sin haber definido antes el problema es una de las causas más frecuentes de proyectos mal orientados. La selección debe partir de preguntas previas: ¿qué se desea predecir o descubrir?, ¿qué tipo de salida se necesita?, ¿existen datos etiquetados?, ¿el problema requiere clasificación, regresión, agrupamiento, reducción de dimensión, detección de anomalías o recomendación?, ¿importa más la precisión, la velocidad o la interpretabilidad?

Cuando el problema está bien formulado, la comparación entre algoritmos deja de ser arbitraria. Por ejemplo, si se quiere predecir una cantidad continua, como el consumo energético, el problema es de regresión. Si se quiere asignar una etiqueta entre varias categorías, como “fraude” o “no fraude”, se trata de clasificación. Si no hay etiquetas y se pretende encontrar grupos naturales, el enfoque será no supervisado.

### 2.2. Tipos de tareas y correspondencia con familias de algoritmos.

Cada familia de algoritmos responde mejor a determinadas estructuras de problema. Los modelos lineales suelen funcionar bien cuando las relaciones entre variables son relativamente simples y se necesita interpretabilidad. Los árboles de decisión y sus variantes capturan relaciones no lineales y manejan bien datos tabulares. Los métodos basados en distancia, como k-NN, pueden resultar útiles en conjuntos pequeños o medianos, aunque sufren cuando la dimensionalidad aumenta mucho. Los modelos de redes neuronales destacan cuando hay patrones complejos, datos no estructurados o grandes volúmenes.

| Tipo de tarea          | Objetivo                   | Algoritmos habituales                                              | Comentario                                           |
|------------------------|----------------------------|--------------------------------------------------------------------|------------------------------------------------------|
| Clasificación          | Asignar clases             | Regresión logística, árboles, random forest, SVM, redes neuronales | Adecuado cuando la salida es una etiqueta.           |
| Regresión              | Predecir valores continuos | Regresión lineal, árboles de regresión, boosting, redes neuronales | Importa medir error numérico.                        |
| Clustering             | Encontrar grupos           | k-means, DBSCAN, clustering jerárquico                             | No requiere etiquetas previas.                       |
| Reducción de dimensión | Resumir variables          | PCA, t-SNE, UMAP, autoencoders                                     | Útil para compresión, visualización o preprocesado.  |
| Detección de anomalías | Encontrar casos raros      | Isolation Forest, LOF, autoencoders                                | Muy usada en fraude, ciberseguridad y mantenimiento. |

### 2.3. Factores que deben guiar la elección del algoritmo.

La conveniencia de un algoritmo no depende solo del tipo de tarea. También depende de las características de los datos y de las condiciones del proyecto. Un profesional competente analiza, al menos, seis grupos de factores: tamaño del conjunto de datos, calidad y distribución de las variables, necesidad de explicación, recursos de cómputo disponibles, tiempo máximo de respuesta y coste del error.

Por ejemplo, cuando el conjunto de datos es pequeño y se necesita una justificación clara de las decisiones, puede ser razonable comenzar con modelos sencillos y explicables. En cambio, cuando el volumen de datos es masivo y la señal es muy compleja, puede convenir un enfoque más potente, aunque exija infraestructura adicional.

- La **interpretabilidad** es clave en sanidad, finanzas o administración pública.
- La **latencia** es decisiva en aplicaciones en tiempo real, como recomendadores o detección de fraude transaccional.
- La **capacidad de escalar** condiciona el trabajo con grandes volúmenes de datos o flujos continuos.
- La **sensibilidad al desbalanceo** importa mucho cuando una clase es muy minoritaria, como en fraude o averías.
- La **facilidad de mantenimiento** también cuenta: un sistema demasiado complejo puede ser inviable a medio plazo.

## 2.4. Modelos simples frente a modelos complejos.

Existe una tentación frecuente de asumir que un modelo más complejo siempre será mejor. Esto no es cierto. Un modelo complejo puede ajustarse mejor a los datos de entrenamiento, pero también puede sobreajustarse y generalizar peor. Los modelos simples tienen ventajas muy valiosas: son más rápidos, más baratos de entrenar, más fáciles de explicar y, en muchos conjuntos tabulares, pueden competir sorprendentemente bien con soluciones más sofisticadas.

La práctica profesional recomienda comenzar con una **línea base**. Una línea base es un modelo sencillo que sirve como referencia mínima. No tiene sentido desplegar una solución muy compleja si apenas mejora a la línea base o si esa mejora no compensa el coste adicional.

## 2.5. Redes neuronales frente a modelos clásicos.

Las redes neuronales son muy potentes, pero no deben elegirse por moda. Su uso está especialmente justificado cuando hay grandes cantidades de datos, relaciones muy no lineales, o cuando se trabaja con texto, audio, imagen o secuencias complejas. En cambio, para muchos problemas tabulares de empresa, modelos como random forest, gradient o regresión logística pueden ofrecer resultados excelentes con menor coste y mayor interpretabilidad.

Elegir bien implica reconocer que cada herramienta tiene un contexto natural. Un profesional maduro sabe que la “mejor” técnica no es universal, sino situacional.

### Criterio de selección

Un algoritmo es conveniente cuando **resuelve bien el problema, se ajusta a los datos disponibles, cumple las restricciones del entorno, y puede justificarse técnica y funcionalmente.**

## 2.6. Ejemplo guiado de selección.

Supongamos que una empresa quiere predecir si un cliente abandonará un servicio. Dispone de datos tabulares: antigüedad, tipo de tarifa, incidencias, uso mensual y variables demográficas. La salida es binaria: abandona o no abandona. En este caso, el problema es de clasificación supervisada.

Una estrategia razonable sería comenzar con regresión logística como línea base, porque permite interpretar el peso de las variables. Después podrían probarse árboles, random forest o gradient

para comprobar si la no linealidad mejora el rendimiento. Si la empresa exige explicaciones claras al área comercial, quizá no compense usar un modelo demasiado opaco si la mejora es marginal.

La valoración final no dependería solo de la accuracy. Habría que observar el recall de la clase “abandona”, porque perder clientes reales puede ser más costoso que generar algunas falsas alarmas.

### **3. Principios y técnicas básicas de los sistemas inteligentes.**

#### **3.1. Qué entendemos por sistema inteligente.**

Un sistema inteligente es un sistema informático capaz de percibir información del entorno, procesarla, extraer significado y actuar o recomendar acciones de forma adaptativa. La inteligencia no implica necesariamente conciencia ni razonamiento humano completo. En ingeniería, un sistema se considera inteligente cuando utiliza estrategias que permiten resolver problemas complejos, tomar decisiones, aprender de la experiencia o aproximarse a metas bajo condiciones cambiantes.

Dentro de esta definición caben sistemas muy diferentes: desde un clasificador de correos no deseados hasta un recomendador de productos, un detector de anomalías en registros industriales o un asistente conversacional. Lo importante es comprender que estos sistemas combinan representación de información, inferencia, aprendizaje, optimización y evaluación.

#### **3.2. Representación del conocimiento y modelización del problema.**

Un principio básico de la inteligencia artificial es que el sistema solo puede operar sobre una representación del problema. En otras palabras, antes de aprender o inferir, la máquina necesita que la realidad se traduzca a un formato computable. En machine learning esto suele materializarse en variables, características, etiquetas, vectores o matrices.

Representar bien el problema es decisivo. Si una tarea se formula con variables pobres o irrelevantes, el mejor algoritmo del mundo obtendrá un rendimiento mediocre. Por eso la ingeniería de características, la selección de variables y la comprensión del dominio son elementos centrales de la práctica.

#### **3.3. Búsqueda, heurística e inferencia.**

Muchos sistemas inteligentes funcionan explorando espacios de posibilidades. A veces esa exploración es explícita, como en algoritmos de búsqueda; otras veces es implícita, como en la optimización de parámetros. La heurística aparece cuando no es viable explorar todas las opciones y se necesitan reglas que orienten la búsqueda hacia soluciones prometedoras.

La inferencia, por su parte, consiste en obtener conclusiones a partir de información disponible. En sistemas basados en reglas, la inferencia puede derivar nuevas afirmaciones. En modelos probabilísticos, la inferencia permite estimar la probabilidad de distintos estados o clases. En aprendizaje automático, inferir significa aplicar un modelo entrenado para obtener una predicción.

#### **3.4. Aprendizaje y adaptación.**

La característica distintiva del aprendizaje automático es que el sistema no se limita a ejecutar reglas escritas a mano, sino que ajusta sus parámetros a partir de datos. Aprender implica encontrar regularidades que permitan hacer predicciones o descubrir estructura. Ese aprendizaje puede ser supervisado, no supervisado, semisupervisado o por refuerzo, aunque en este módulo dominan especialmente los enfoques supervisado y no supervisado.

La adaptación significa que el sistema cambia su comportamiento en función de la experiencia. Cuanto mejor se representa el problema, más apropiado es el algoritmo y más riguroso es el proceso de validación, más probable es que esa adaptación conduzca a un comportamiento útil.

### 3.5. Incertidumbre y probabilidad.

Los sistemas inteligentes trabajan casi siempre en presencia de incertidumbre. Los datos pueden ser incompletos, ruidosos o ambiguos, y el mundo real rara vez ofrece certezas absolutas. Por ello, muchas técnicas se apoyan en la probabilidad y la estadística. Un clasificador no suele afirmar “esto es fraude” con certeza total, sino con una probabilidad o puntuación.

Entender esta idea es fundamental para valorar resultados. Las decisiones no deben basarse en una fe ciega en el modelo, sino en la interpretación de sus salidas, en la confianza asociada y en el coste potencial del error.

### 3.6. Optimización como corazón del aprendizaje.

La mayoría de los modelos aprenden mediante procesos de optimización. Entrenar consiste en buscar los parámetros que minimizan una función de error o pérdida. Aunque el alumnado utilice bibliotecas que ocultan buena parte del detalle matemático, debe comprender que detrás de un “fit” existe un problema de optimización donde se equilibran ajuste, generalización y coste.

Este punto conecta directamente con la valoración de calidad. Un modelo puede haber alcanzado un mínimo muy bueno sobre el entrenamiento y, sin embargo, generalizar mal. Por eso optimizar no es solo reducir error en entrenamiento; es también evitar que el sistema memorice patrones accidentales.

Evaluar la aplicación práctica de los principios y técnicas de los sistemas inteligentes significa reconocer **cómo intervienen la representación, la inferencia, la búsqueda, la optimización, el aprendizaje y la gestión de la incertidumbre** en una solución concreta.

## 4. Integración de principios fundamentales de la computación en proyectos de IA.

### 4.1. La IA no sustituye los fundamentos de la informática.

Uno de los errores conceptuales más extendidos consiste en tratar la inteligencia artificial como un campo separado de la computación. En realidad, la IA aplicada se apoya continuamente en fundamentos clásicos de la informática. Sin estructuras de datos adecuadas, sin abstracción, sin modularidad, sin análisis de complejidad y sin buenas prácticas de programación, los proyectos de IA se vuelven frágiles, difíciles de mantener y poco escalables.

### 4.2. Abstracción y descomposición del problema.

La abstracción consiste en centrarse en los aspectos relevantes del problema y omitir los detalles que no aportan valor al modelo. En aprendizaje automático, abstraer implica decidir qué entradas son significativas, cómo definir la variable objetivo y qué componentes del sistema deben separarse.

La descomposición, por su parte, permite dividir una solución en módulos. Un sistema de IA bien diseñado suele separar claramente la ingestión de datos, el preprocesamiento, el entrenamiento,

la validación, la persistencia del modelo, la inferencia y la monitorización. Esta separación facilita la reutilización, el mantenimiento y las pruebas.

### 4.3. Algoritmos y estructuras de datos.

Todo modelo de aprendizaje automático termina ejecutándose sobre estructuras de datos concretas: arrays, matrices dispersas, dataframes, colas, tablas hash, árboles, grafos o almacenamientos distribuidos. Elegir la estructura adecuada repercute en el rendimiento. Un mal diseño puede disparar el consumo de memoria o ralentizar enormemente el procesamiento.

El alumnado debe comprender que la eficiencia no es un detalle secundario. Cuando el volumen de datos crece, las elecciones de representación y acceso se convierten en factores decisivos.

### 4.4. Complejidad temporal y espacial.

La complejidad temporal mide cómo crece el tiempo de ejecución al aumentar el tamaño de la entrada. La complejidad espacial mide el crecimiento del consumo de memoria. En proyectos con grandes volúmenes de datos, estas dos dimensiones condicionan la viabilidad de una técnica.

Por ejemplo, algoritmos basados en distancias entre todos los pares de ejemplos pueden resultar costosos cuando el conjunto de datos es enorme. Del mismo modo, una red neuronal profunda puede ofrecer gran capacidad representacional, pero exigir hardware específico y tiempos de entrenamiento largos.

Valorar algoritmos implica, por tanto, pensar en términos computacionales: no solo qué resultado generan, sino cuánto cuesta obtenerlo.

### 4.5. Reproducibilidad, versionado y calidad del software.

En IA profesional, reproducir un experimento es tan importante como obtenerlo por primera vez. Un proceso reproducible permite repetir el entrenamiento con las mismas condiciones y comprobar que se obtienen resultados equivalentes. Para ello es necesario controlar versiones de código, dependencias, semillas aleatorias, conjuntos de datos y configuraciones.

La calidad del software también exige pruebas. Aunque el modelo sea estadístico, el sistema que lo rodea es software y debe verificarse. Se prueban transformaciones de datos, validaciones de entrada, serialización de modelos, APIs, procesos batch y monitorización. Una predicción correcta con datos bien formados no sirve de nada si la aplicación falla cuando recibe un valor nulo o un formato inesperado.

### 4.6. Escalabilidad y procesamiento distribuido.

Cuando los datos alcanzan tamaños elevados, resulta insuficiente pensar en un único equipo o en procesos manuales. Aparecen entonces principios de escalabilidad horizontal, paralelización, particionado y procesamiento distribuido. Esto es especialmente relevante en el contexto del curso de especialización, donde IA y Big Data se integran de forma natural.

La selección tecnológica debe considerar si un algoritmo puede entrenarse por lotes, si admite paralelización, si funciona bien en entornos distribuidos o si necesita simplificarse para ser viable.

### 4.7. Seguridad, privacidad y robustez.

Los principios fundamentales de la computación también obligan a pensar en seguridad y robustez. Los datos pueden contener información sensible, y una canalización de IA puede convertirse en un punto crítico del sistema. Es necesario proteger el acceso a los datos, registrar operaciones, controlar permisos y diseñar mecanismos que eviten fallos silenciosos.



Además, un sistema robusto es aquel que no colapsa ante entradas anómalas, valores ausentes, formatos inesperados o cambios moderados en la distribución de los datos. Diseñar robustez es una tarea de ingeniería, no una casualidad.

## 5. Desarrollo de sistemas y aplicaciones informáticas que incorporan técnicas inteligentes.

### 5.1. Del experimento al sistema.

En el aula es habitual entrenar modelos en cuadernos de trabajo o notebooks. Sin embargo, un experimento aislado no equivale a una aplicación informática. Desarrollar una solución basada en IA implica encapsular el modelo dentro de un sistema que gestiona datos, reglas de negocio, interfaces, almacenamiento, seguridad y mantenimiento.

### 5.2. Arquitectura básica de una aplicación con IA.

Una arquitectura típica puede dividirse en varias capas. La primera es la capa de datos, donde se almacenan, validan y transforman las entradas. La segunda es la capa de modelo, donde reside el artefacto entrenado y la lógica de inferencia. La tercera es la capa de servicio, que expone el modelo mediante una API, un proceso batch o un módulo interno. La cuarta es la capa de consumo, donde una aplicación, panel o proceso automático utiliza la predicción para tomar decisiones.

Separar estas capas mejora la mantenibilidad. Permite sustituir un modelo por otro, actualizar transformaciones o cambiar la interfaz sin rehacer todo el sistema.

### 5.3. Tipos de integración.

Las técnicas inteligentes pueden integrarse de formas muy distintas. En algunos casos, el modelo actúa de forma offline, generando predicciones periódicas para un informe. En otros, responde en tiempo real, como en una API que clasifica documentos en el momento de la carga. También puede funcionar como motor de apoyo a la decisión, mostrando una recomendación que después valida una persona.

La forma de integración afecta a la evaluación de calidad. Un modelo aceptable para análisis nocturno por lotes puede ser inadecuado para una aplicación interactiva si tarda varios segundos en responder.

### 5.4. MLOps básico: despliegue, monitorización y actualización.

Un sistema con IA no termina cuando se despliega. Tras la puesta en marcha hay que monitorizar su comportamiento. Se observan métricas de uso, tiempos de respuesta, errores técnicos, deriva de datos, pérdida de rendimiento y necesidad de reentrenamiento. Esta visión operativa recibe a menudo el nombre de MLOps y conecta el aprendizaje automático con la ingeniería de software y la administración de sistemas.

Para el alumnado es importante comprender que los modelos envejecen. Un clasificador entrenado con datos de hace dos años puede dejar de ser válido si cambian los hábitos de los usuarios, la estacionalidad del negocio o el modo en que se generan los registros.



## 5.5. Ejemplo de aplicación: clasificación automática de incidencias.

Imaginemos un servicio técnico que recibe miles de incidencias en lenguaje natural. El sistema puede incorporar una técnica de clasificación de texto para etiquetar automáticamente cada incidencia: red, hardware, software, acceso o facturación.

La solución completa incluiría un formulario o correo de entrada, un preprocesamiento del texto, un modelo entrenado, una API que devuelva la categoría predicha, un panel de supervisión y un registro de las decisiones tomadas. La calidad ya no se reduce a la precisión del clasificador. También importa el tiempo de respuesta, la trazabilidad y la capacidad de corrección manual.

| Componente             | Función principal                          | Riesgo si se diseña mal                      |
|------------------------|--------------------------------------------|----------------------------------------------|
| Ingesta de datos       | Recibir y validar entradas                 | Datos inconsistentes o pérdida de registros. |
| Preprocesamiento       | Limpiar, transformar y codificar           | Sesgos, fuga de información o incoherencias. |
| Modelo                 | Generar predicciones o agrupaciones        | Bajo rendimiento o sobreajuste.              |
| Servicio de inferencia | Exponer la predicción al resto del sistema | Latencia alta o errores de integración.      |
| Monitorización         | Vigilar rendimiento y deriva               | Degradación no detectada en producción.      |

### Aplicación frente a demostración

Una **demostración** enseña que un modelo funciona en un entorno controlado. Una **aplicación informática** demuestra que ese modelo puede utilizarse de forma sostenida, integrada y segura en una necesidad real.

## 6. Técnicas de aprendizaje computacional orientadas a la extracción automática de información en grandes volúmenes de datos.

### 6.1. Qué significa extraer información automáticamente.

Extraer información automáticamente no es simplemente almacenar muchos datos. Significa transformar datos brutos en conocimiento útil sin intervención manual continua. Ese conocimiento puede adoptar muchas formas: clases, patrones, segmentos de usuarios, anomalías, tendencias, temas dominantes en documentos, recomendaciones o alertas.

En grandes volúmenes de datos, la automatización deja de ser una comodidad y se convierte en una necesidad. Ningún equipo humano puede revisar millones de registros, mensajes o eventos con la velocidad requerida. La técnica computacional adecuada permite detectar regularidades y señalar la información relevante.

## 6.2. Fases de una canalización de extracción automática.

La extracción automática suele organizarse como una canalización o pipeline. Primero se recopilan datos desde fuentes heterogéneas: bases de datos, ficheros, sensores, logs, formularios, APIs o documentos. Después se realiza una limpieza para tratar valores perdidos, formatos inconsistentes y duplicidades. A continuación se transforman las variables y, si procede, se generan nuevas características. Finalmente se entrena o aplica un modelo que identifica patrones y produce salidas útiles.

En entornos masivos, estas fases deben diseñarse para ejecutarse con eficiencia. Eso implica procesado por lotes, paralelización, uso de almacenamiento adecuado y control de trazabilidad.

## 6.3. Preparación de datos: la etapa más determinante.

En la práctica, gran parte del éxito de un proyecto depende de la preparación de datos. Los datos crudos rara vez están listos para alimentar un modelo. Pueden contener ruido, valores faltantes, variables irrelevantes, categorías mal codificadas o escalas incompatibles.

Preparar bien implica limpiar, normalizar, codificar variables categóricas, detectar outliers, equilibrar clases si es necesario y garantizar que las transformaciones del entrenamiento se apliquen igual en producción. Una evaluación aparentemente excelente puede ser engañosa si se ha producido fuga de información durante esta etapa.

## 6.4. Ingeniería de características y selección de variables.

La ingeniería de características consiste en construir representaciones más útiles a partir de los datos disponibles. Puede implicar combinar variables, extraer agregados temporales, calcular frecuencias, obtener medias móviles, crear indicadores binarios o resumir textos mediante vectores.

La selección de variables, por su parte, busca conservar la información relevante y eliminar ruido o redundancia. Trabajar con demasiadas variables irrelevantes puede empeorar el rendimiento, aumentar el coste y dificultar la interpretación.

## 6.5. Extracción de información en texto.

El texto es una fuente inmensa de información empresarial y social: correos, incidencias, reseñas, publicaciones, contratos o historiales. Para extraer conocimiento de texto se aplican técnicas como clasificación documental, análisis de sentimiento, detección de temas, extracción de entidades y similitud semántica.

Aunque los métodos modernos pueden ser muy avanzados, el principio sigue siendo el mismo: convertir el lenguaje natural en una representación computable que permita detectar patrones relevantes.

## 6.6. Extracción de información en registros y eventos.

Otra situación muy común es el análisis de logs y eventos generados por sistemas informáticos, sensores o transacciones. Aquí resultan especialmente útiles la detección de anomalías, el análisis secuencial y la agregación temporal. Estas técnicas permiten identificar comportamientos extraños, caídas de servicio, intentos de intrusión, averías o desviaciones operativas.

El valor de estas aplicaciones es alto porque convierten millones de eventos aparentemente inconexos en señales de alerta accionables.

### 6.7. Aprendizaje supervisado y no supervisado en grandes volúmenes.

En extracción automática de información se combinan con frecuencia técnicas supervisadas y no supervisadas. Las primeras son útiles cuando existen etiquetas históricas y se quiere predecir una salida concreta. Las segundas son valiosas cuando se persigue descubrir estructura interna: grupos, segmentos, relaciones ocultas o casos raros.

Un ejemplo clásico es el análisis de clientes. Un algoritmo de clustering puede descubrir segmentos de comportamiento, y después un modelo supervisado puede predecir la probabilidad de abandono en cada segmento.

### 6.8. Escalabilidad y valor de negocio.

Cuanto mayor es el volumen de datos, más importante se vuelve el equilibrio entre sofisticación y viabilidad. No siempre es recomendable aplicar el modelo más complejo; a veces conviene una solución que procese datos de manera estable y rápida, incluso si pierde algo de precisión frente a alternativas costosas.

## 7. Validación, capacidad de generalización, test y control del sobreajuste.

### 7.1. La capacidad de generalización.

La capacidad de generalización es uno de los conceptos más importantes de toda la unidad. Un modelo generaliza bien cuando mantiene un rendimiento satisfactorio con datos que no ha visto durante el entrenamiento. Esto es fundamental, porque el objetivo real no es recordar el pasado, sino responder correctamente ante nuevos casos.

Cuando un modelo aprende patrones genuinos del fenómeno, suele generalizar. Cuando memoriza detalles accidentales del conjunto de entrenamiento, aparece el sobreajuste. Y cuando el modelo es demasiado simple para captar la estructura del problema, surge el subajuste.

### 7.2. Conjuntos de entrenamiento, validación y test.

Para evaluar adecuadamente un modelo es habitual dividir los datos en varios subconjuntos. El conjunto de entrenamiento se utiliza para ajustar parámetros. El conjunto de validación sirve para comparar configuraciones, ajustar hiperparámetros y tomar decisiones durante el desarrollo. El conjunto de test se reserva para la evaluación final, una vez que las decisiones principales ya se han tomado.

La separación es esencial. Si el conjunto de test influye en la elección del modelo, deja de ser una medida honesta del rendimiento final. Por eso suele decirse que el test debe permanecer “intocable” hasta el final.

### 7.3. Hold-out y validación cruzada.

La estrategia más sencilla de validación es el hold-out: separar un porcentaje de datos para validación y otro para test. Sin embargo, cuando el conjunto de datos es limitado, puede ser preferible usar validación cruzada. En la validación cruzada, el conjunto se divide en varias particiones y el entrenamiento se repite dejando cada partición como validación una vez.

Este procedimiento ofrece una estimación más estable del rendimiento medio y reduce la dependencia de una sola división de datos. Es especialmente útil para comparar algoritmos cuando no se dispone de un conjunto muy grande.

## 7.4. Sobreajuste e subajuste

El sobreajuste aparece cuando el modelo se ajusta en exceso a peculiaridades del conjunto de entrenamiento y pierde capacidad para generalizar. Se observa a menudo cuando la métrica de entrenamiento es excelente, pero la de validación o test cae de forma clara.

El subajuste, en cambio, surge cuando el modelo ni siquiera aprende suficientemente del entrenamiento. Suele deberse a una capacidad insuficiente del modelo, a características poco informativas o a un proceso de entrenamiento inadecuado.

El profesional debe distinguir ambos fenómenos, porque sus soluciones son distintas. El sobreajuste puede mitigarse con regularización, simplificación del modelo, más datos o mejor selección de variables. El subajuste puede requerir modelos más ricos, más información o entrenamiento más adecuado.

## 7.5. Fuga de información o data leakage.

La fuga de información es uno de los errores más peligrosos en evaluación. Ocurre cuando datos o transformaciones que deberían permanecer fuera del entrenamiento influyen indirectamente en él. Por ejemplo, si se normaliza todo el conjunto completo antes de separar entrenamiento y test, la información del test ya ha contaminado el proceso.

La fuga produce métricas artificialmente optimistas. Por eso cualquier transformación aprendida a partir de los datos debe ajustarse solo con el conjunto de entrenamiento y aplicarse después al resto.

## 7.6. Curvas de aprendizaje y diagnóstico del modelo.

Las curvas de aprendizaje comparan el rendimiento del modelo en entrenamiento y validación conforme aumenta el tamaño del conjunto o el número de iteraciones. Son muy útiles para diagnosticar si conviene añadir datos, cambiar de modelo o ajustar regularización.

Si ambas curvas son bajas y cercanas, puede haber subajuste. Si la de entrenamiento es muy buena y la de validación claramente peor, puede existir sobreajuste. Este tipo de lectura ayuda a tomar decisiones más informadas que la simple observación de una métrica final.

## 7.7. Qué papel tiene el test.

El conjunto de test no debe usarse para experimentar repetidamente hasta encontrar el mejor resultado. Su función es aportar una fotografía final, creíble y externa del rendimiento esperado. Cuando el proyecto ya ha elegido modelo, hiperparámetros y pipeline, entonces sí se consulta el test para estimar la calidad final.

| Conjunto      | Para qué se usa                                    | Qué error debe evitarse                                  |
|---------------|----------------------------------------------------|----------------------------------------------------------|
| Entrenamiento | Ajustar parámetros del modelo                      | Pensar que su métrica refleja rendimiento real.          |
| Validación    | Comparar configuraciones y ajustar hiperparámetros | Sobreoptimizar decisiones sobre el mismo conjunto.       |
| Test          | Evaluación final del modelo elegido                | Usarlo durante el desarrollo y contaminar la evaluación. |

Un modelo de calidad no es el que “mejor recuerda” los datos de entrenamiento, sino el que **generaliza de forma consistente** cuando se enfrenta a situaciones nuevas.

## 8. Métricas de evaluación y lectura de la matriz de confusión.

### 8.1. Por qué una sola métrica casi nunca basta.

La evaluación cuantitativa es necesaria, pero puede ser engañosa si se resume en un único número. En muchos problemas, distintas métricas captan aspectos distintos del rendimiento. Por eso un análisis sólido combina medidas y las interpreta a la luz del contexto.

En un conjunto muy desbalanceado, por ejemplo, una accuracy alta puede ocultar que el modelo no detecta casi ningún caso positivo. Del mismo modo, en regresión un error medio aceptable puede esconder errores enormes en ciertos segmentos.

### 8.2. La matriz de confusión.

La matriz de confusión es una herramienta central para tareas de clasificación. Resume cuántos ejemplos de cada clase han sido clasificados correctamente o incorrectamente. En el caso binario aparecen cuatro cantidades fundamentales: verdaderos positivos, falsos positivos, verdaderos negativos y falsos negativos.

Los verdaderos positivos son casos positivos correctamente detectados. Los falsos positivos son casos negativos que el sistema ha marcado erróneamente como positivos. Los verdaderos negativos son negativos bien clasificados. Los falsos negativos son positivos que el sistema no ha detectado.

| Real \ Predicho | Positivo                | Negativo                |
|-----------------|-------------------------|-------------------------|
| Positivo        | Verdadero positivo (TP) | Falso negativo (FN)     |
| Negativo        | Falso positivo (FP)     | Verdadero negativo (TN) |

La utilidad de la matriz de confusión es que permite entender **qué tipo de error** comete el modelo. No es lo mismo generar falsas alarmas que dejar escapar casos críticos. Dependiendo de la aplicación, uno de esos errores puede ser mucho más costoso que el otro.

### 8.3. Accuracy, precisión, recall, especificidad y F1.

A partir de la matriz de confusión se calculan métricas clave. La accuracy mide la proporción total de aciertos, pero no distingue qué clases se están favoreciendo. La precisión indica, de entre los casos predichos como positivos, cuántos lo eran realmente. El recall indica, de entre los positivos reales, cuántos ha logrado encontrar el modelo.

La especificidad mide la capacidad de reconocer correctamente los negativos. La F1 combina precisión y recall en una media armónica, resultando útil cuando interesa equilibrar ambos aspectos.

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

$$\text{Precision} = TP / (TP + FP)$$

$$\text{Recall} = TP / (TP + FN)$$

$$\text{Specificity} = TN / (TN + FP)$$

$$F1 = 2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall})$$

### 8.4. Elección de métricas según el contexto.

No existe una métrica universalmente superior. En detección de enfermedad, suele importar mucho el recall, porque dejar sin detectar un caso real puede tener consecuencias graves. En filtrado de spam, quizá se acepte perder algún correo no deseado si con ello se evita enviar correos legítimos a la carpeta de spam, lo que pone foco en precisión y especificidad.

La selección de métricas es, en sí misma, una decisión profesional. Demuestra que la evaluación se orienta al propósito real del sistema.

### 8.5. Curvas ROC, AUC y umbrales de decisión.

Muchos clasificadores no producen solo una etiqueta, sino una probabilidad o puntuación. Para convertir esa puntuación en clase es necesario fijar un umbral. Cambiar el umbral modifica el equilibrio entre falsos positivos y falsos negativos.

La curva ROC representa la relación entre sensibilidad y tasa de falsos positivos para distintos umbrales. El área bajo la curva, o AUC, resume la capacidad del modelo para separar clases. En problemas muy desbalanceados también conviene atender a curvas de precisión-recall.

### 8.6. Métricas de regresión.

Cuando la salida es numérica, la evaluación cambia. En regresión resultan habituales métricas como MAE, MSE, RMSE y  $R^2$ . El MAE expresa el error medio absoluto y suele ser fácil de interpretar. El MSE penaliza con más fuerza los errores grandes. El RMSE devuelve la raíz cuadrada del error cuadrático medio y conserva la unidad de la variable. El coeficiente  $R^2$  mide qué proporción de variabilidad explica el modelo.

$MAE = \text{promedio de } |\text{valor\_real} - \text{valor\_predicho}|$

$MSE = \text{promedio de } (\text{valor\_real} - \text{valor\_predicho})^2$

$RMSE = \text{raíz cuadrada del MSE}$

$R^2 = 1 - (\text{error\_residual} / \text{error\_total})$

### 8.7. Métricas para clustering y aprendizaje no supervisado.

En aprendizaje no supervisado la evaluación es más compleja, porque a menudo no existen etiquetas de referencia. En clustering pueden emplearse medidas internas, como el índice silhouette, que evalúa compactación y separación de grupos, o criterios como Davies-Bouldin. También puede analizarse la utilidad práctica de los grupos obtenidos: si son interpretables, si ayudan a segmentar acciones y si permanecen estables.

### 8.8. Evaluación cualitativa y análisis de errores.

Junto a las métricas numéricas, conviene examinar ejemplos concretos de aciertos y fallos. Este análisis cualitativo ayuda a descubrir patrones de error, sesgos, ambigüedades en las etiquetas o carencias en los datos.

#### No medir solo, sino interpretar

Una métrica nunca habla por sí sola. El alumnado debe aprender a **leer el significado del resultado**, a relacionarlo con el coste del error y a decidir si el comportamiento observado es suficiente para el problema real.

## 9. Interpretación de resultados, explicabilidad y toma de decisiones.

### 9.1. De la métrica a la decisión técnica.

Una vez obtenidos los resultados, el siguiente paso consiste en decidir qué hacer con ellos. Esta fase exige juicio profesional. A veces el resultado indica que el modelo puede desplegarse. Otras veces sugiere que conviene recoger más datos, redefinir variables, ajustar umbrales o incluso replantear el problema.

La interpretación correcta combina métricas, análisis de errores, coste computacional y contexto de uso. La decisión final debe justificarse de forma argumentada.

### 9.2. Explicabilidad e interpretabilidad.

La explicabilidad se refiere a la capacidad de entender o justificar por qué el modelo produce ciertas salidas. En algunos casos esa explicación se obtiene directamente porque el modelo es interpretable, como ocurre con reglas sencillas, árboles pequeños o regresión lineal/logística. En otros, se recurre a técnicas de explicación local o global.

La necesidad de explicabilidad no es igual en todos los ámbitos, pero cada vez es más importante. Un sistema que toma decisiones con impacto sobre personas debe poder revisarse, discutirse y, en su caso, corregirse.

### 9.3. Sesgo, equidad y evaluación responsable.

Valorar la calidad también implica preguntarse si el modelo se comporta de forma injusta con determinados grupos o si amplifica sesgos presentes en los datos históricos. Un modelo técnicamente preciso puede ser problemático si perjudica sistemáticamente a una categoría de usuarios.

La evaluación responsable incluye revisar distribuciones, balance de datos, representatividad y diferencias de rendimiento entre subgrupos. La IA de calidad no es solo eficaz; también debe ser razonable, auditable y alineada con criterios éticos y legales.

### 9.4. Cuándo un modelo está listo para producción.

No existe una cifra mágica que indique cuándo un modelo está listo para entrar en producción. La decisión depende de si cumple el umbral mínimo exigido por el problema, si sus errores están controlados, si el sistema que lo rodea es estable y si existe capacidad de monitorización posterior.

En muchas organizaciones se utilizan listas de comprobación: rendimiento mínimo, validación reproducible, tiempos aceptables, documentación técnica, plan de actualización y protocolo de supervisión humana.

| Pregunta                                      | Sí/No esperado | Por qué importa                                         |
|-----------------------------------------------|----------------|---------------------------------------------------------|
| ¿Se ha validado con datos no vistos?          | Sí             | Sin esta condición no se conoce la generalización real. |
| ¿Las métricas elegidas responden al problema? | Sí             | Evita tomar decisiones con indicadores inadecuados.     |



| Pregunta                                          | Sí/No esperado | Por qué importa                                     |
|---------------------------------------------------|----------------|-----------------------------------------------------|
| ¿El tiempo de inferencia es asumible?             | Sí             | Un sistema lento puede ser inviable aunque acierte. |
| ¿Existe monitorización y plan de reentrenamiento? | Sí             | El rendimiento puede degradarse en producción.      |
| ¿Se han analizado riesgos y sesgos?               | Sí             | La calidad incluye responsabilidad y robustez.      |

### 9.5. Documentar para poder defender.

Un buen desarrollo de IA no solo debe funcionar; también debe poder explicarse. Documentar el problema, los datos usados, el pipeline, las métricas, las comparaciones y las limitaciones del modelo permite defender técnicamente el trabajo. Esta habilidad es especialmente valiosa en contextos académicos y profesionales.

#### La competencia que se busca

El alumnado debe ser capaz de explicar qué significa el resultado, qué errores comete el modelo, qué limitaciones tiene, cuánto cuesta operarlo y si sería adecuado implantarlo.

## 10. Casos prácticos guiados.

### 10.1. Caso 1: detección de spam en correos electrónicos

**Problema:** una organización desea clasificar automáticamente los correos entrantes como spam o legítimos. Dispone de miles de ejemplos históricos etiquetados. El problema es de clasificación supervisada.

**Selección de algoritmo:** pueden plantearse como línea base modelos sencillos de clasificación de texto, y compararse después con alternativas más complejas. La conveniencia dependerá del volumen de datos, de la necesidad de rapidez y del equilibrio entre precisión y recall.

**Evaluación:** en este caso no basta con la accuracy. Un exceso de falsos positivos puede enviar correos legítimos a la carpeta de spam, con un impacto muy negativo. Conviene revisar precisión, recall y matriz de confusión. También es útil ajustar el umbral de decisión.

**Conclusión técnica:** el mejor modelo será el que mantenga un buen filtrado sin dañar de forma apreciable la recepción de mensajes válidos y pueda integrarse en tiempo real en el servidor o aplicación de correo.

### 10.2. Caso 2: segmentación de clientes mediante clustering.

**Problema:** una empresa de comercio electrónico quiere identificar grupos de clientes con comportamientos de compra similares para personalizar campañas. No dispone de etiquetas previas, así que el enfoque adecuado es no supervisado.

Selección de algoritmo: se pueden comparar técnicas de clustering como k-means, clustering jerárquico o DBSCAN. La elección dependerá de la forma de los grupos, del número esperado de segmentos y de la sensibilidad al ruido.

Evaluación: además de medidas internas como silhouette, hay que valorar si los grupos obtenidos tienen sentido de negocio. Un clustering técnicamente correcto pero imposible de interpretar aporta poco valor.

Conclusión técnica: el algoritmo conveniente será aquel que produzca segmentos estables, separados y accionables para marketing o atención al cliente.

### 10.3. Caso 3: detección de anomalías en logs de un sistema.

Problema: un equipo de operaciones quiere detectar automáticamente eventos extraños en millones de líneas de log para anticipar fallos o intrusiones. Las etiquetas son escasas, por lo que puede optarse por detección de anomalías o enfoques híbridos.

Selección de algoritmo: resultan adecuados modelos centrados en la rareza del patrón, agregaciones temporales e incluso autoencoders si el volumen y la complejidad lo justifican. Pero no debe perderse de vista el coste computacional ni la facilidad de operar en tiempo casi real.

Evaluación: interesa medir cuántas alertas útiles se generan frente a cuántas falsas alarmas aparecen. En operaciones, un sistema con demasiados falsos positivos puede dejar de usarse por fatiga de alertas.

Conclusión técnica: la mejor solución es la que identifica eventos anómalos relevantes, con latencia asumible y una tasa de falsas alertas tolerable para el equipo humano.

#### Lo que muestran los casos

Los tres casos ponen de relieve la misma idea: **la evaluación nunca es abstracta**. Siempre depende del tipo de dato, del coste del error, de la necesidad de explicabilidad y del contexto de integración.

## 11. Buenas prácticas y errores frecuentes.

### 11.1. Buenas prácticas recomendables.

- Definir con precisión la variable objetivo y el propósito de la solución antes de elegir algoritmos.
- Construir una **línea base** simple y explicable antes de pasar a modelos complejos.
- Separar de forma rigurosa entrenamiento, validación y test.
- Aplicar todas las transformaciones del pipeline evitando fuga de información.
- Elegir métricas alineadas con el coste real del error.
- Comparar modelos no solo por rendimiento, sino también por eficiencia, interpretabilidad y escalabilidad.
- Documentar decisiones, parámetros, versiones y limitaciones.
- Monitorizar el modelo una vez desplegado.

### 11.2. Errores frecuentes que deben evitarse.

- Escoger un algoritmo por moda o por familiaridad, sin analizar si encaja con el problema.

- Celebrar una accuracy alta en conjuntos desbalanceados sin revisar recall, precisión o matriz de confusión.
- Usar el conjunto de test para tomar decisiones de desarrollo.
- No detectar sobreajuste porque solo se observa el rendimiento en entrenamiento.
- Olvidar que el sistema completo incluye validación de entradas, servicio, latencia y mantenimiento.
- Pensar que más complejidad siempre implica más calidad.
- Confundir rendimiento estadístico con valor real para la organización.
- Ignorar sesgos, deriva de datos o problemas de robustez.

### Error conceptual muy habitual

Obtener una métrica elevada no demuestra por sí solo que el sistema sea bueno. La calidad exige **contexto, honestidad metodológica e integración con principios de computación**.

## Síntesis.

Valorar la calidad de los resultados en aprendizaje automático exige una mirada amplia. Hay que elegir algoritmos con criterio, comprender los principios de los sistemas inteligentes, integrar fundamentos de computación, desarrollar aplicaciones completas y utilizar técnicas capaces de extraer información útil de grandes volúmenes de datos.

La evaluación rigurosa se apoya en la capacidad de generalización, la separación entre entrenamiento, validación y test, el control del sobreajuste y la lectura adecuada de métricas como la matriz de confusión. Sin embargo, la verdadera competencia profesional aparece cuando estos elementos se utilizan para tomar decisiones justificadas.

En definitiva, un sistema de IA de calidad no es el que impresiona más, sino el que resuelve un problema real con suficiente fiabilidad, coste asumible, robustez, posibilidad de explicación y capacidad de mantenerse útil en el tiempo.

### Importante

Aprender esta unidad no consiste en memorizar nombres de algoritmos. Consiste en desarrollar criterio para **comparar, justificar, validar, interpretar y desplegar** soluciones de aprendizaje automático con fundamento técnico.